Comunicaciones en red

Programación en red

La programación en redes solía ser una tarea compleja, ya que el desarrollador tenía que comprender numerosos detalles técnicos relacionados con la red, incluyendo el hardware utilizado, los diferentes niveles de la capa de red y las bibliotecas necesarias para cada nivel.

Sin embargo, en esencia, la comunicación en red se reduce a intercambiar información entre dos máquinas a través de aplicaciones. Este intercambio de datos puede simplificarse a una operación similar a leer y escribir en archivos, aunque con algunas diferencias específicas.

El sistema de entrada/salida en Unix sigue un modelo que se conoce como Abrir-Leer-Escribir-Cerrar. Para que un proceso pueda realizar estas operaciones, primero debe abrir (open) el archivo o dispositivo deseado, obteniendo los permisos necesarios.

Una vez abierto, el proceso puede realizar lecturas (read) o escrituras (write) para transferir datos entre el sistema y el usuario. Al finalizar el intercambio de datos, se debe cerrar (close) el archivo o dispositivo para notificar al sistema operativo que ya no se usará.

En Unix, los procesos disponen de descriptores de entrada y salida que permiten estas operaciones. Estos descriptores pueden representar archivos, dispositivos o canales de comunicación como los sockets.

En el caso de los sockets, su ciclo de vida se divide en tres etapas principales:

- Creación: Se establece el socket.
- Lectura y escritura: Se envían y reciben datos a través del socket.
- Cierre: Se cierra el socket al terminar la comunicación.

La interfaz IPC (comunicación entre procesos) en Unix-BSD utiliza los protocolos TCP y UDP. Las direcciones de socket identifican a los destinatarios, combinando una dirección IP con un número de puerto.

El intercambio de información ocurre entre pares de sockets, transmitiendo datos en forma de mensajes. Los mensajes se almacenan temporalmente en el socket emisor hasta que se envían y, en el receptor, hasta que el proceso correspondiente los lee.

En cuanto al lenguaje Java, fue creado por Sun Microsystems en la década de 1990 bajo la dirección de James Gosling. Inicialmente, el objetivo era desarrollar un sistema para controlar dispositivos electrónicos y asistentes personales (PDAs) con capacidad de conexión a redes. Este proyecto también implicaba la creación de hardware versátil, un sistema operativo eficiente (SunOS) y un lenguaje de

programación llamado Oak. Aunque el proyecto terminó en un fracaso y el equipo fue disuelto, Java surgió posteriormente como un lenguaje clave para el desarrollo de aplicaciones en red.

El desarrollo del proyecto relacionado con el lenguaje Oak continuó adelante, en gran parte, gracias a su distribución gratuita a través de Internet, aprovechando la naciente World Wide Web de aquellos años. Esto permitió que el lenguaje ganara popularidad y atrajera a numerosos programadores, quienes contribuyeron a mejorarlo, depurarlo y definir sus características y usos principales.

El nombre "Java" surgió durante una de las sesiones de lluvia de ideas organizadas por el equipo de desarrollo. El cambio de nombre fue necesario porque ya existía otro lenguaje llamado Oak.

Sun Microsystems presentó las primeras versiones de Java a principios de 1995. Desde entonces, se han lanzado continuamente nuevas versiones que han ampliado las funcionalidades del lenguaje y fomentado su adopción en el desarrollo de aplicaciones.

Una de las cualidades más destacadas de Java es su sencillez y potencia para crear aplicaciones orientadas a redes. Los aspectos técnicos de bajo nivel están abstraídos y gestionados de forma transparente por la Máquina Virtual de Java (JVM). Esto garantiza que los programas sean independientes de la arquitectura del sistema y puedan ejecutarse en una amplia gama de dispositivos con diferentes procesadores y sistemas operativos.

CLASES PARA LAS COMUNICACIONES DE RED EN JAVA: java.net

En las aplicaciones de red, es común utilizar el modelo cliente-servidor. En este paradigma, el servidor se encarga de esperar conexiones desde los clientes en una ubicación específica, mientras que el cliente envía solicitudes al servidor ejecutándose en una máquina y escucha en un puerto determinado. Una vez establecida la conexión, esta se gestiona como un flujo típico de entrada y salida.

Cuando se desarrollan programas en Java que interactúan a través de la red, generalmente se trabaja en la capa de aplicación. Esto significa que no es necesario lidiar directamente con las capas TCP y UDP, ya que las clases del paquete java.net abstraen estos detalles, proporcionando comunicación de red independiente de la plataforma.

Las clases del paquete java.net permiten a los programas Java utilizar tanto TCP como UDP para establecer comunicación en la red:

TCP: Ofrece un canal de comunicación confiable y punto a punto, ideal para aplicaciones cliente-servidor.Clases principales: Socket y ServerSocket, que implementan el lado cliente y servidor respectivamente.
UDP: Permite la transmisión de datos sin conexión, útil para aplicaciones que requieren menor sobrecarga en la comunicación.Clases principales: DatagramSocket y DatagramPacket.
A continuación, se destacan las principales clases del paquete java.net que facilitan la implementación de comunicaciones en red:

- Socket: Representa un extremo de una conexión TCP.

- ServerSocket: Gestiona el lado servidor de la conexión, esperando solicitudes de los clientes.
- DatagramSocket: Proporciona funcionalidad tanto para el cliente como para el servidor cuando se utiliza UDP.
- DatagramPacket: Maneja los paquetes de datos para servicios de red sin conexión, permitiendo el envío y la recepción de paquetes.
- InetAddress: Es una clase diseñada para manejar direcciones IP en aplicaciones de red.

Estas clases son herramientas clave para desarrollar aplicaciones en red con Java y serán exploradas en detalle en los apartados siguientes.

La clase Socket del paquete java.net implementa una solución multiplataforma para establecer un canal de comunicación bidireccional entre un cliente y un servidor. Gracias a java.net.Socket, los desarrolladores pueden gestionar conexiones de red sin depender del código nativo del sistema, permitiendo que las aplicaciones Java mantengan su independencia de la plataforma.

El entorno de desarrollo Java también incluye el paquete java.io, que proporciona una variedad de herramientas para gestionar canales de entrada y salida. Dentro de este paquete, las clases abstractas InputStream y OutputStream definen la estructura básica de los flujos de datos para lectura y escritura en Java.

Además, java.io contiene múltiples subclases de InputStream y OutputStream que ofrecen implementaciones específicas para diferentes tipos de operaciones de entrada y salida, adaptándose a las necesidades concretas de cada aplicación.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Modelo cliente-servidor

Modelo Cliente-Servidor: una arquitectura fundamental en la comunicación en redes

El modelo cliente-servidor es la forma más común de comunicación en redes, especialmente en Internet. Su propósito principal es habilitar y proporcionar una amplia variedad de servicios a los usuarios, desde la navegación web y el acceso a bases de datos hasta la transmisión de video y el correo electrónico. Esta arquitectura se basa en la interacción entre dos entidades clave: el cliente y el servidor, cada una con roles específicos y características definidas.

El cliente

El cliente es el proceso que inicia la comunicación. Es el responsable de:

1. Enviar una solicitud:
   El cliente se conecta al servidor utilizando una dirección y un puerto específicos, que debe conocer de antemano. Esta dirección puede ser una IP o un nombre de dominio, mientras que el puerto identifica el servicio que se solicita (por ejemplo, HTTP usa el puerto 80 por defecto).
2. Recibir la respuesta:
   Una vez que el cliente envía la solicitud, espera la respuesta del servidor, que puede incluir datos, mensajes de error o confirmaciones.
3. Proceso finito:
   Aunque el cliente puede enviar múltiples solicitudes y recibir varias respuestas en una sesión, todo el proceso es finito. Es decir, la conexión entre el cliente y el servidor eventualmente termina, ya sea porque el cliente cierra la sesión o porque el servicio solicitado se completa.

Un ejemplo clásico de cliente es un navegador web que solicita páginas web desde un servidor, como cuando ingresamos una URL en la barra de direcciones.

El servidor

El servidor, en cambio, tiene un papel pasivo pero crucial. Es un proceso que:

1. Espera solicitudes:
   El servidor no inicia la comunicación, sino que permanece en espera de que los clientes lo contacten. Esto lo convierte en un elemento esencial en la red, ya que es el encargado de procesar las solicitudes y enviar las respuestas correspondientes.

2. Proporciona servicios:
   Una vez que el servidor recibe una solicitud, la procesa y proporciona al cliente la información o servicio solicitado. Esto puede incluir datos almacenados, resultados de cálculos o la ejecución de una acción.
3. Proceso infinito:
   A diferencia del cliente, el servidor está diseñado para funcionar de manera continua. Permanece activo y disponible para atender nuevas solicitudes indefinidamente, hasta que se apague manualmente o sufra algún fallo.
4. Modos de funcionamiento:
   - Iterativo: procesa una solicitud a la vez, lo que puede ser suficiente para servicios con pocas solicitudes simultáneas.
   - Concurrente: maneja múltiples solicitudes al mismo tiempo, lo cual es crucial para servicios de alta demanda, como páginas web populares o aplicaciones de streaming.

Un ejemplo clásico de servidor es un servidor web como Apache o Nginx, que gestiona las solicitudes de múltiples navegadores.

Relación Cliente-Servidor

La interacción entre el cliente y el servidor tiene ciertas características clave:

1. El cliente debe conocer al servidor:
   Antes de iniciar la comunicación, el cliente necesita conocer la dirección del servidor (IP o nombre de dominio) y el puerto donde se ofrece el servicio deseado. Sin esta información, el cliente no puede establecer la conexión.
2. El servidor no necesita conocer al cliente previamente:
   El servidor está diseñado para aceptar solicitudes de cualquier cliente que cumpla con los requisitos de acceso (como credenciales o permisos). No necesita saber de antemano quién se conectará, lo que lo hace flexible y escalable.
3. Comunicación basada en solicitudes y respuestas:
   Todo el modelo gira en torno a este intercambio básico: el cliente envía una solicitud, y el servidor responde. Esto puede repetirse múltiples veces en una sesión, pero sigue siendo una comunicación bien definida y delimitada.

Ventajas del Modelo Cliente-Servidor

1. Separación de responsabilidades:
   El cliente se enfoca en solicitar servicios y presentar resultados, mientras que el servidor se encarga del almacenamiento, procesamiento y entrega de datos.
2. Escalabilidad:
   Los servidores pueden ser configurados para manejar miles de clientes concurrentemente,

especialmente cuando se utilizan servidores concurrentes.

3. Estandarización:
   Los protocolos como HTTP, FTP y SMTP se basan en este modelo, lo que permite una interoperabilidad amplia entre diferentes sistemas.
4. Facilidad de mantenimiento:
   Las actualizaciones o cambios en el servidor no requieren modificaciones en los clientes, siempre que se mantenga la compatibilidad del protocolo.

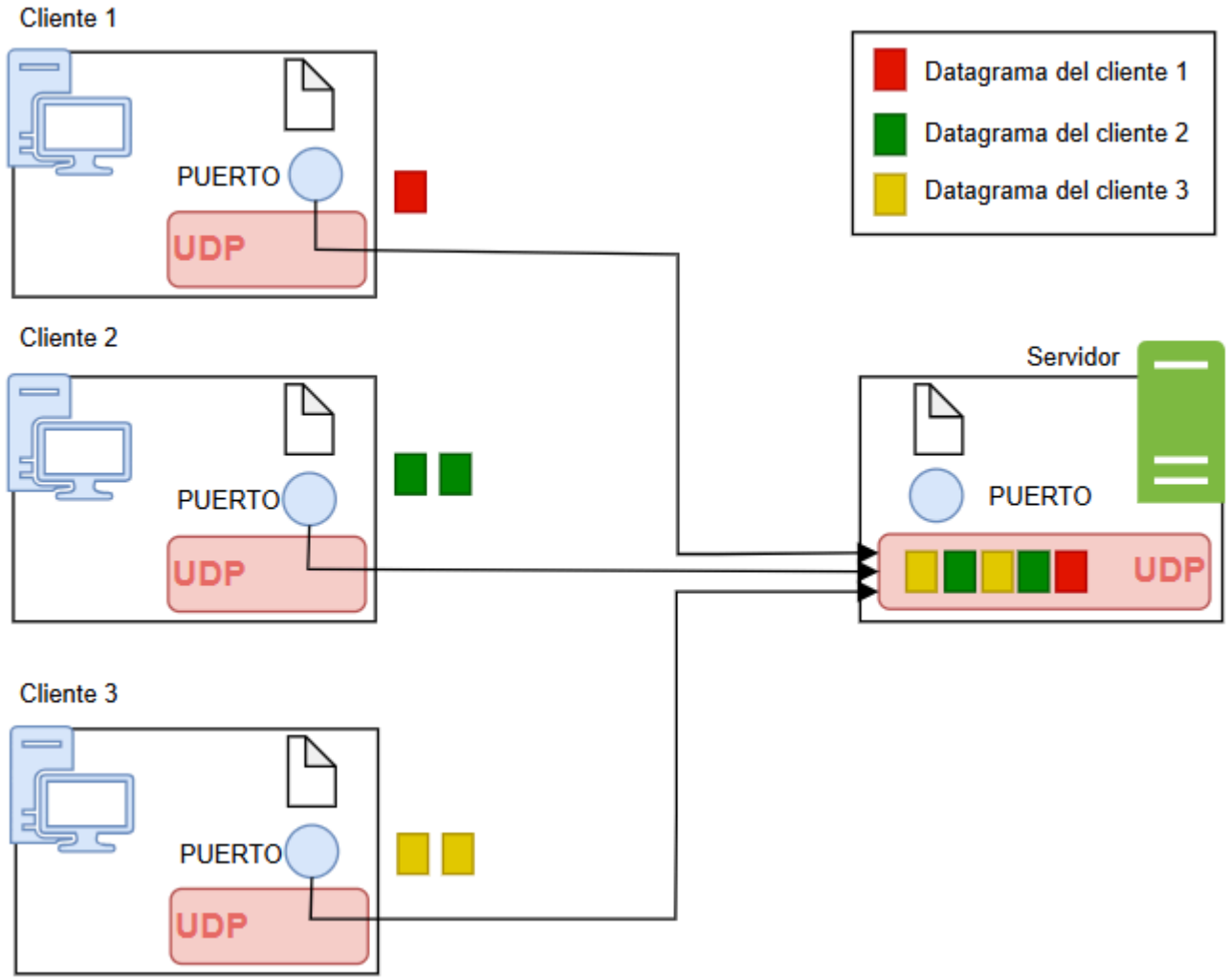
Ejemplos del Modelo Cliente-Servidor

- Navegación web: el cliente (navegador) solicita una página web al servidor (servidor web) y recibe los datos para mostrarlos al usuario.
- Correo electrónico: el cliente (como Outlook) solicita nuevos correos del servidor (servidor de correo) y permite al usuario enviar o leer mensajes
- Aplicaciones de streaming: el cliente (aplicación móvil o web) solicita contenido multimedia al servidor, que lo envía en paquetes para su reproducción.

El modelo cliente-servidor es una de las bases más importantes de las tecnologías modernas, y entender sus principios permite desarrollar y gestionar servicios de red de manera eficiente.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Comunicación usando UDP



La imagen representa un esquema del funcionamiento de la comunicación entre múltiples clientes y un servidor utilizando el protocolo UDP (User Datagram Protocol). Aquí está la explicación detallada de los elementos y el flujo representado:

1. **Clientes (Client 1, Client 2, Client 3):**
   - Cada cliente está representado como un dispositivo (computadora o aplicación) que envía datos al servidor.
   - Cada cliente utiliza un puerto efímero (Ephemeral port), que es un puerto asignado dinámicamente para enviar datos. Este puerto es temporal y se utiliza únicamente durante la comunicación actual.
2. **Servidor**:
   - El servidor está configurado para escuchar en un puerto bien conocido (Well-known port). Este puerto es fijo y conocido por los clientes, permitiendo que las aplicaciones se conecten al servicio ofrecido.

- El servidor tiene una cola de entrada (Incoming queue) donde se almacenan temporalmente los datagramas entrantes de los clientes antes de ser procesados.

3. **UDP (User Datagram Protocol):**
   - Es un protocolo de transporte que se utiliza para enviar datos desde los clientes al servidor.
   - UDP es un protocolo sin conexión, lo que significa que los datagramas (paquetes de datos) se envían directamente sin establecer una conexión previa entre el cliente y el servidor.
   - No garantiza la entrega de los datos, el orden de llegada, ni verifica si hubo pérdidas, lo que lo hace más rápido pero menos confiable que TCP.

Flujo de comunicación:

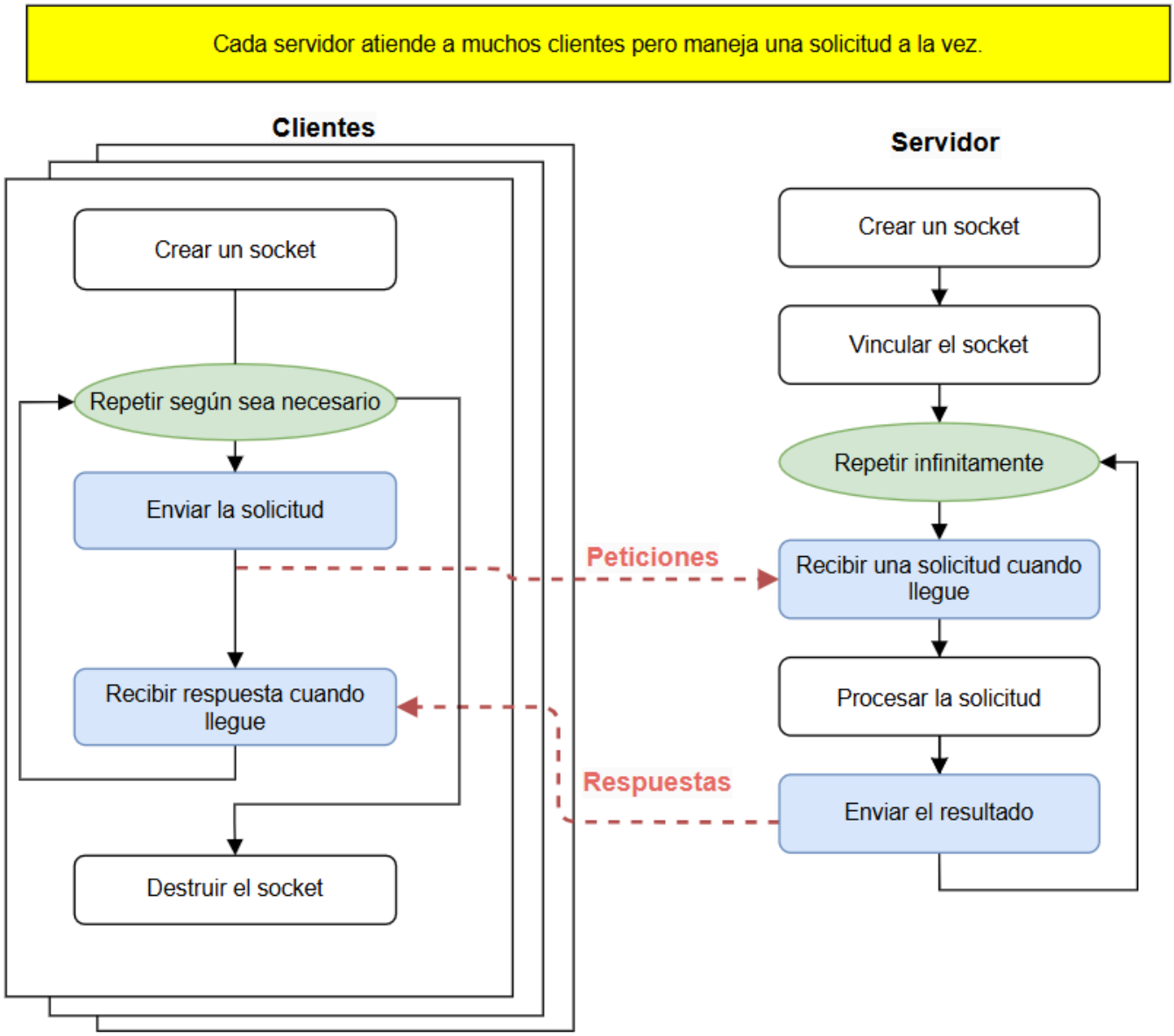
1. Clientes envían datagramas:
   - Cada cliente utiliza su puerto efímero para enviar datagramas al servidor.
   - Los datagramas son pequeños paquetes de datos que contienen información como la dirección IP del cliente, el puerto efímero, el puerto destino (servidor) y los datos a transmitir.
2. Llegada al servidor:
   - Los datagramas enviados por los clientes llegan al puerto bien conocido del servidor, que está configurado específicamente para recibir este tipo de comunicación.
3. Cola de entrada del servidor:
   - Los datagramas entrantes se almacenan temporalmente en la cola de entrada del servidor en el orden en que llegan.
   - El servidor procesará estos datagramas uno por uno según su disponibilidad.

Características clave del modelo UDP

1. Sin conexión:
   - No se establece una conexión formal entre el cliente y el servidor antes de enviar los datos. Esto permite una comunicación rápida, pero significa que no hay confirmación de que los datos hayan llegado correctamente.
2. Multiplicidad de clientes:
   - Varios clientes pueden comunicarse con el mismo servidor al mismo tiempo utilizando sus puertos efímeros.
   - El servidor distingue los clientes utilizando las direcciones IP y los puertos efímeros incluidos en los datagramas.
3. Velocidad sobre confiabilidad:
   - UDP es más rápido que TCP porque no tiene mecanismos de retransmisión ni control de flujo. Sin embargo, esto significa que si un datagrama se pierde o llega fuera de orden, el servidor no

tiene manera de recuperarlo.

Principios de la Comunicación Cliente-Servidor con UDP



PRINCIPIOS DE COMUNICACIÓN CLIENTE-SERVIDOR

Lado del Cliente (Izquierda):

Flujo de acciones del cliente:

Crear un socket:
El cliente inicia el proceso estableciendo un socket, que será el medio de comunicación con el servidor. Esto incluye asignar un puerto efímero y especificar la dirección del servidor (IP y puerto).

Repetir según sea necesario:
Dentro de un ciclo, el cliente puede realizar múltiples

Lado del Servidor (Derecha):

Flujo de acciones del servidor:

Crear un socket:
El servidor inicializa un socket que estará vinculado a un puerto conocido. Este socket espera conexiones entrantes de los clientes.

Vincular el socket:
Se asocia el socket del servidor con una

solicitudes al servidor mientras sea necesario. Esto permite que el cliente interactúe de manera continua con el servidor.

Enviar la solicitud:
El cliente utiliza el socket para enviar datos o solicitudes específicas al servidor. Por ejemplo, pedir una página web o enviar datos a una base de datos.

Recibir respuesta cuando llegue:
Una vez que el servidor procesa la solicitud, el cliente espera y recibe la respuesta. Esto puede ser una confirmación, datos solicitados, o un mensaje de error.

Destruir el socket:
Cuando ya no se necesitan más interacciones, el cliente cierra y libera el socket. Esto finaliza la conexión de manera ordenada.

dirección IP y un puerto específico, lo que permite que los clientes sepan cómo contactarlo.

Repetir infinitamente:
El servidor entra en un bucle continuo para estar disponible y recibir solicitudes en cualquier momento.

Recibir una solicitud cuando llegue:
El servidor espera solicitudes entrantes de los clientes en el socket. Cuando llega una solicitud, la procesa de manera secuencial.

Procesar la solicitud:
El servidor interpreta la solicitud y realiza la acción requerida, como devolver datos, guardar información, o realizar cálculos.

Enviar el resultado:
Una vez que la solicitud es procesada, el servidor envía una respuesta al cliente a través del socket.

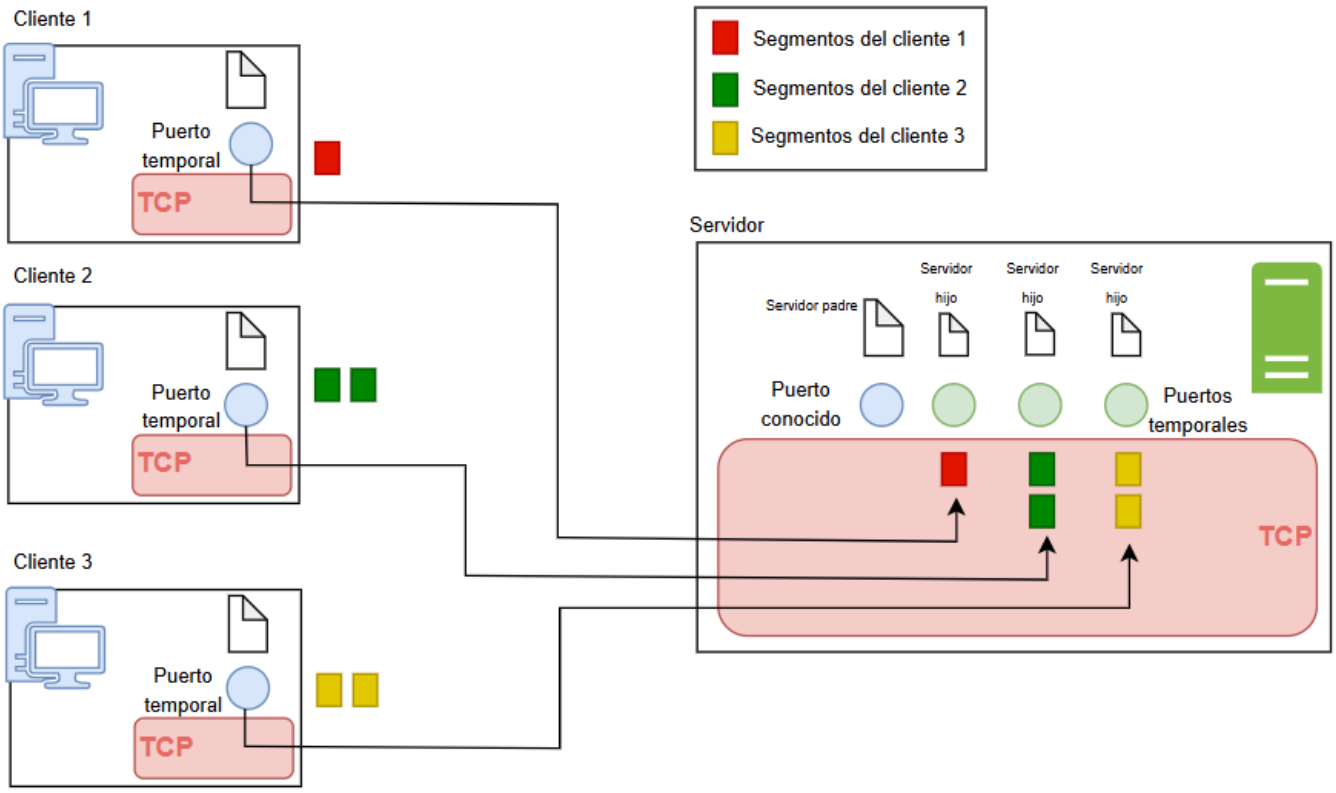
Interacción entre Cliente y Servidor (Conexión):

Peticiones (línea punteada roja hacia la derecha):
Estas son las solicitudes enviadas por los clientes hacia el servidor. Ejemplo: un cliente pide un archivo o consulta una base de datos.

Respuestas (línea punteada roja hacia la izquierda):
Son las respuestas del servidor hacia los clientes. Ejemplo: el servidor devuelve el archivo solicitado o confirma que los datos fueron procesados correctamente.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Comunicación usando TCP



La imagen ilustra cómo funciona un servidor concurrente utilizando el protocolo TCP (Transmission Control Protocol) para gestionar múltiples conexiones simultáneamente. Aquí está una explicación detallada de los elementos clave y el flujo de funcionamiento:

Elementos principales:

1. Clientes (Client 1, Client 2, Client 3):
   - Cada cliente es un dispositivo o proceso que utiliza un puerto efímero (Ephemeral Port) para establecer la comunicación con el servidor.TCP es el protocolo utilizado por los clientes para enviar segmentos de datos al servidor. Los segmentos se identifican por colores:
     - Magenta: segmento del Cliente 1.
     - Verde: segmento del Cliente 2.
     - Amarillo: segmento del Cliente 3.
2. Servidor principal (Parent Server):
   - El servidor principal está configurado para escuchar en un puerto bien conocido (Well-known Port), que es fijo y conocido por los clientes. Este puerto es utilizado solo para establecer la conexión inicial entre el cliente y el servidor.

- Después de aceptar una conexión, el servidor crea un servidor hijo (Child Server) que se encarga de gestionar exclusivamente esa conexión.

3. Servidores hijos (Child Servers):
   - Una vez que el servidor principal establece la conexión inicial, delega la comunicación con cada cliente a un servidor hijo.
   - Los servidores hijos utilizan sus propios puertos efímeros para gestionar las comunicaciones con los clientes de manera individual y simultánea.
   - Esto permite que el servidor sea concurrente, ya que puede manejar múltiples clientes al mismo tiempo sin interferencia.
4. Cola de entrada (Incoming Queues):
   - Cada cliente tiene su propia cola de entrada en el servidor. Estas colas almacenan temporalmente los segmentos de datos que llegan antes de ser procesados por el servidor correspondiente.

Flujo de comunicación:

1. Conexión inicial:
   - Cada cliente utiliza un puerto efímero para enviar una solicitud de conexión al puerto bien conocido del servidor principal.
   - El servidor principal acepta la conexión y crea un servidor hijo para gestionar esa comunicación.
2. Gestión concurrente:
   - Cada servidor hijo gestiona exclusivamente la comunicación con un cliente específico. Esto permite que varios clientes puedan enviar y recibir datos simultáneamente sin afectar el rendimiento del sistema.
   - Los segmentos de datos de cada cliente (identificados por colores) se procesan independientemente en sus respectivas colas de entrada.
3. Transmisión de datos:
   - Los clientes envían segmentos de datos al servidor, que son colocados en las colas de entrada correspondientes.
   - Los servidores hijos leen los datos de sus colas, los procesan y envían respuestas al cliente.

Características del servidor concurrente

1. Procesamiento de múltiples solicitudes:
   - Un servidor concurrente puede gestionar muchas solicitudes al mismo tiempo gracias a la creación de servidores hijos, cada uno atendiendo a un cliente.

2. Protocolo TCP:
    - TCP es un protocolo orientado a conexión, lo que significa que se establece una conexión confiable antes de enviar los datos.
    - Asegura que los datos lleguen correctamente, en orden, y sin duplicados.
3. Separación de tareas:
    - El servidor principal se encarga solo de aceptar nuevas conexiones.
    - Los servidores hijos se encargan de procesar las solicitudes y respuestas, asegurando que cada cliente tenga su propio canal de comunicación.

Nota clave:

- "El puerto bien conocido se utiliza únicamente para el establecimiento de la conexión."
  Esto significa que después de que se crea una conexión, la comunicación real se lleva a cabo a través de los puertos efímeros asignados a los servidores hijos y no a través del puerto bien conocido.
- "Los servidores que utilizan TCP suelen ser concurrentes."
  Debido a la necesidad de gestionar múltiples conexiones simultáneamente, los servidores que implementan TCP generalmente son concurrentes para mantener la eficiencia y la confiabilidad.

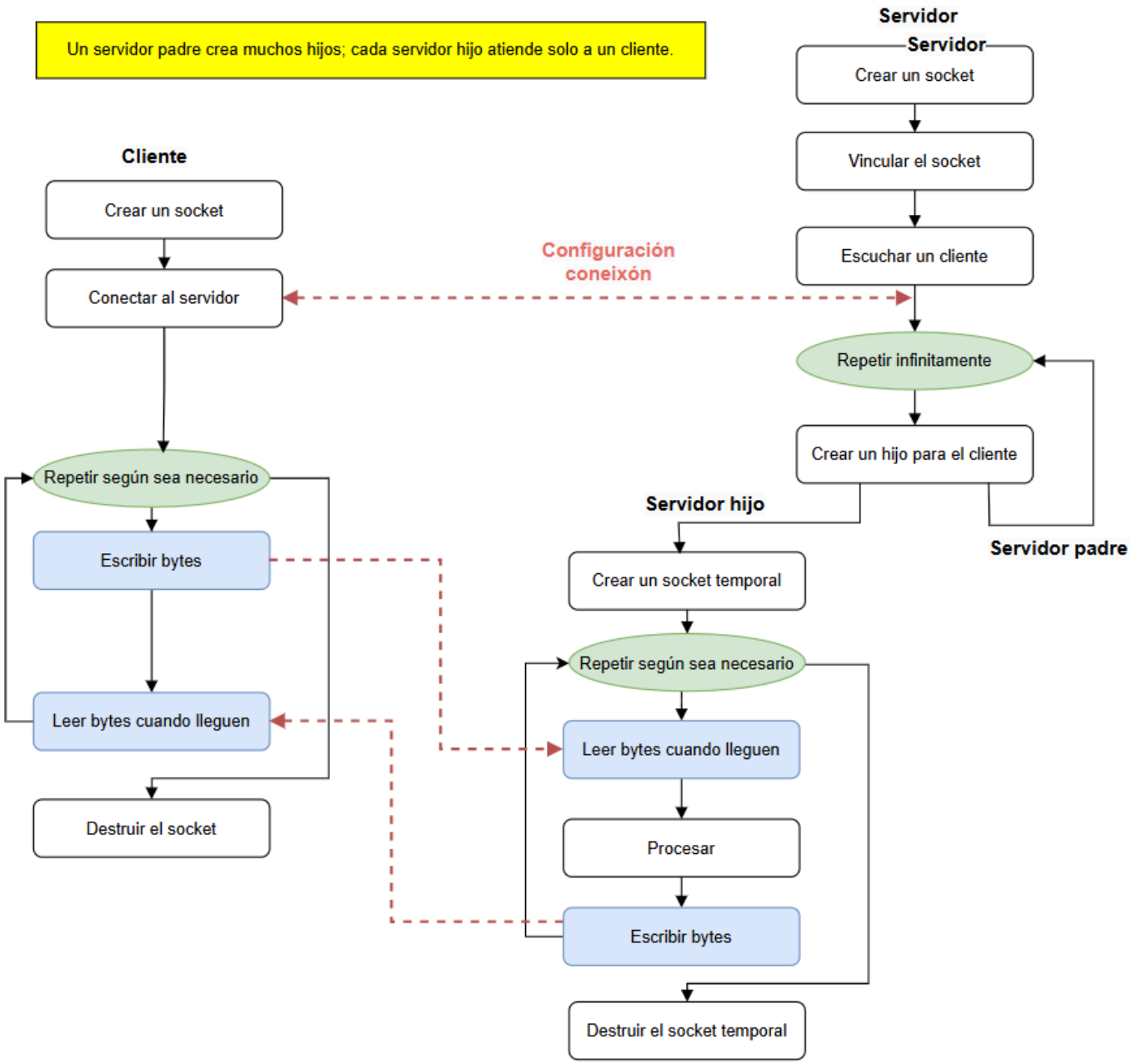
Aplicaciones prácticas:

- Navegación web: los navegadores utilizan TCP para descargar páginas web desde servidores concurrentes.
- Servicios de mensajería: aplicaciones como WhatsApp o Telegram utilizan este enfoque para manejar mensajes simultáneamente.
- Servidores FTP: permiten múltiples transferencias de archivos al mismo tiempo.

Esta arquitectura es fundamental para la mayoría de los servicios modernos en Internet, ya que asegura tanto la escalabilidad como la eficiencia en el manejo de múltiples conexiones.

Principios de la comunicación usando TCP

La imagen representa el flujo de trabajo en un modelo cliente-servidor utilizando sockets en una arquitectura donde el servidor principal (padre) crea múltiples servidores hijos para gestionar a los clientes de forma concurrente. Este enfoque asegura que cada cliente sea atendido por un servidor hijo específico, permitiendo múltiples interacciones simultáneas.



MODELO CLIENTE-SERVIDOR CON SERVIDORES HIJOS

Cliente

1. Crear un socket: El cliente inicializa un socket para comunicarse con el servidor.

2. Conectar al servidor: Establece la conexión con el servidor principal.

3. Repetir según sea necesario: Puede realizar múltiples intercambios de datos.

4. Escribir bytes: Envía datos al servidor.

Servidor padre

1. Crear un socket: Inicializa un socket principal.

2. Vincular el socket: Lo asocia con un puerto conocido.

3. Escuchar un cliente: Permanece en espera de conexiones.

4. Repetir infinitamente: El servidor principal está siempre disponible.

5. Crear un hijo para el cliente:

Servidor hijo

1. Crear un socket temporal: Inicializa un socket exclusivo para el cliente.

2. Repetir según sea necesario: Maneja múltiples intercambios de datos.

3. Leer bytes cuando lleguen: Recibe datos del cliente.

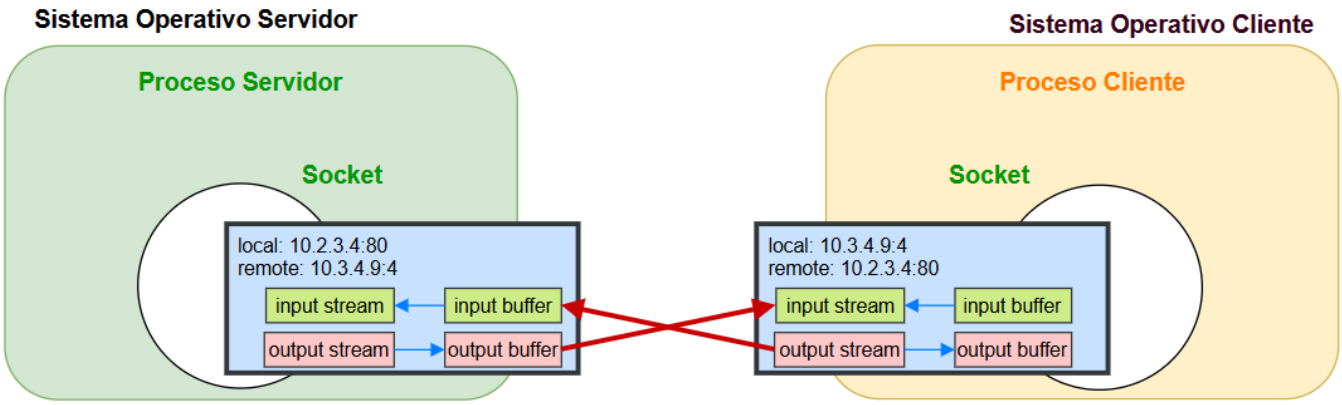
4. Procesar: Realiza las acciones requeridas con los datos recibidos.

5. Leer bytes cuando lleguen: Recibe respuestas del servidor. **6. Destruir el socket:** Finaliza la conexión cerrando el socket.	Genera un servidor hijo para manejar la conexión.	**5. Escribir bytes:** Envía la respuesta al cliente. **6. Destruir el socket temporal:** Cierra el socket al finalizar.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Comunicación a través de flujos

La imagen ilustra cómo funciona la comunicación entre un cliente y un servidor utilizando sockets en el contexto de sistemas operativos. A continuación, se describe cada parte y su funcionamiento:



1. **Proceso del Servidor (Server Process):**
   El proceso en el servidor es el encargado de atender las solicitudes de los clientes. Este proceso está asociado a un socket que actúa como punto de comunicación con los clientes.
   - **Socket del Servidor:**
     El socket en el lado del servidor está identificado por:
     - Local: 10.2.3.4:80
       La dirección IP (10.2.3.4) y el puerto (80) del servidor, que son conocidos y utilizados por el cliente para establecer la conexión. El puerto 80, por ejemplo, es el puerto predeterminado para el protocolo HTTP.
     - Remote: 10.3.4.9:4
       La dirección IP y el puerto efímero del cliente (10.3.4.9:4), que identifica al cliente durante la comunicación.
   - Buffers de Entrada y Salida:
     - Input Stream (Flujo de entrada): Recibe datos enviados desde el cliente. Los datos entrantes se almacenan temporalmente en el buffer de entrada antes de ser procesados por el servidor.
     - Output Stream (Flujo de salida): Envía datos desde el servidor hacia el cliente. Los datos que se van a enviar primero se colocan en el buffer de salida.
2. **Proceso del Cliente (Client Process):**
   Este proceso se encarga de iniciar la comunicación con el servidor enviando solicitudes y recibiendo respuestas.
   - Socket del Cliente:
     El socket en el lado del cliente está identificado por:

        - Local: 10.3.4.9:4
          La dirección IP y el puerto efímero asignado dinámicamente al cliente (4). Este puerto se utiliza exclusivamente para la comunicación actual.
        - Remote: 10.2.3.4:80
          La dirección IP y el puerto bien conocido del servidor (10.2.3.4:80), que el cliente utiliza para establecer la conexión.
    - Buffers de Entrada y Salida:
        - Input Stream (Flujo de entrada): Recibe datos enviados desde el servidor y los almacena temporalmente en el buffer de entrada antes de que sean procesados por el cliente.
        - Output Stream (Flujo de salida): Envía datos desde el cliente hacia el servidor. Los datos se colocan primero en el buffer de salida.

3. **Comunicación entre Cliente y Servidor**
    - Conexión establecida:
      La conexión entre el cliente y el servidor se realiza a través de los sockets. Cada socket tiene información sobre:
        - Su propia dirección y puerto (Local).
        - La dirección y puerto del socket remoto con el que está comunicándose.
    - Intercambio de datos:
        - Los datos fluyen bidireccionalmente entre los sockets.
            - El cliente puede enviar solicitudes al servidor a través de su flujo de salida (Output Stream), que llegan al flujo de entrada del servidor (Input Stream).
            - El servidor procesa la solicitud y envía una respuesta a través de su flujo de salida, que llega al flujo de entrada del cliente.
    - Buffers:
        - Los buffers de entrada y salida permiten que los datos se almacenen temporalmente durante la transmisión, asegurando que el cliente y el servidor puedan enviar y recibir datos de manera eficiente.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Sockets en Java

Paquete java.net

El paquete java.net es una biblioteca integrada en Java que proporciona clases y funcionalidades para realizar operaciones relacionadas con redes. Es un componente esencial para desarrollar aplicaciones cliente-servidor, realizar comunicaciones mediante sockets y manejar protocolos de alto nivel como HTTP, entre otros.

Este paquete permite a los desarrolladores interactuar con recursos de red de manera eficiente, ya sea para conexiones locales (como sockets en la misma máquina) o globales (a través de Internet).

Por ejemplo, si quisiéramos implementar un Socket en un lenguaje como C, el código podría ser perfectamente el que se muestra a continuación:

Código C




```c
int set_up_socket(u_short port) {
    char myname[MAXHOSTNAME+1]; // Nombre del host (máximo permitido)
    int s; // Descriptor del socket
    struct sockaddr_in sa; // Estructura para la dirección del socket
    struct hostent *he; // Puntero a información del host

    bzero(&sa, sizeof(struct sockaddr_in)); // Inicializar estructura
    gethostname(myname, MAXHOSTNAME); // Obtener el nombre del host local
    he = gethostbyname(myname); // Resolver el nombre del host

    if (he == NULL)
        return (-1);

    sa.sin_family = he->h_addrtype; // Configurar familia de direcciones (IPv4)
    sa.sin_port = htons(port); // Configurar el número de puerto

    if ((s = socket(AF_INET, SOCK_STREAM, 0)) < 0)
        return (-1);

    if (bind(s, &sa, sizeof(sa)) < 0) {
        close(s); // Cerrar socket si falla
```

```
            return (-1);
        }

        listen(s, 3); // Configurar para escuchar conexiones (máximo 3 en cola)
        return (s); // Retornar descriptor del socket
    }
```

En cambio todo esto se puede hacer en Java utilizando la siguiente línea:

Código Java



```
ServerSocket servsock = new ServerSocket(port, backlog, bindAddr);
```

Clase	Descripción
InetAddress	Representa una dirección IP. Puede ser IPv4 o IPv6.
ServerSocket	Se utiliza para crear sockets TCP pasivos en el lado del servidor, que esperan conexiones entrantes.
Socket	Representa un socket TCP activo que permite la comunicación entre cliente y servidor.
DatagramSocket	Permite enviar y recibir paquetes de datos a través de UDP, que es un protocolo sin conexión.
DatagramPacket	Representa un paquete de datagramas. Además de los datos, contiene información de la dirección IP y el puerto. **Se utiliza en UDP.**
MulticastSocket	Subclase de **DatagramSocket**. Se puede usar para enviar y recibir paquetes de/para múltiples usuarios.
URL	Permite representar y manipular una dirección de un recurso web (como *http://example.com*).
HttpURLConnection	Clase para establecer conexiones HTTP y manejar solicitudes y respuestas a servidores web.

Ventajas del paquete java.net:

- **Fácil de usar**:Java abstrae la complejidad de las capas de red y transporte, permitiendo a los desarrolladores centrarse en la lógica de la aplicación.
- **Compatibilidad** con protocolos estándar:Soporta los protocolos más utilizados como TCP, UDP y HTTP.
- **Independencia** de plataforma:Los programas desarrollados con java.net funcionan en cualquier sistema operativo que soporte Java.
- **Escalabilidad:Permite desarrollar aplicaciones desde pequeños programas cliente-servidor hasta complejas aplicaciones distribuidas.**
- **Seguridad**:Integra mecanismos de seguridad para controlar el acceso a recursos de red, como el cortafuegos del gestor de seguridad de Java.

Clase	Descripción
InetAddress	Representa una dirección IP. Implementa la interfaz *Serializable*. Tiene dos subclases finales: **Inet4Address** e **Inet6Address**.
ServerSocket	Socket TCP pasivo (servidor). Se utiliza en el lado del servidor para esperar solicitudes de conexión de los clientes.
Socket	Socket TCP activo. Puede ser utilizado como un punto de comunicación tanto en el lado del cliente como en el del servidor.
DatagramSocket	Socket sin conexión (UDP). Se utiliza para enviar y recibir datagramas (paquetes de datos que son direccionados y enrutados individualmente).
DatagramPacket	Representa un paquete de datagramas. Además de los datos, contiene información de la dirección IP y el puerto. **Se utiliza en UDP.**
MulticastSocket	Subclase de **DatagramSocket**. Se puede usar para enviar y recibir paquetes de/para múltiples usuarios.

Inet4Address

La clase Inet4Address es una subclase de InetAddress en el paquete java.net y se utiliza específicamente para representar direcciones IP en el formato IPv4 (Internet Protocol versión 4). IPv4 es

el esquema de direcciones más común, donde una dirección IP consta de cuatro números separados por puntos (por ejemplo, 192.168.1.1).

La clase Inet4Address no se puede instanciar directamente porque es una subclase interna de InetAddress. En lugar de eso, utilizamos métodos estáticos de InetAddress para obtener instancias de Inet4Address.

Características principales de Inet4Address

1. Representación de direcciones IPv4:
    1. Una dirección IPv4 consta de 32 bits divididos en 4 grupos de 8 bits (octetos).
    2. Se utiliza para identificar dispositivos en una red.
2. Uso de la clase Inet4Address:
    1. Es una implementación concreta de InetAddress para trabajar exclusivamente con direcciones IPv4.
    2. Proporciona métodos para resolver nombres de host a direcciones IP y viceversa.
3. Serialización:
    1. Implementa la interfaz Serializable, lo que permite que los objetos de esta clase se serialicen y deserialicen.

Métodos importantes (heredados de InetAddress)

Método	Descripción
getByName(String host)	Resuelve un nombre de host (como *google.com*) en una dirección IP.
getAllByName(String host)	Devuelve todas las direcciones IP asociadas a un nombre de host.
getLocalHost()	Obtiene la dirección IP de la máquina local.
getHostAddress()	Devuelve la representación de la dirección IP en formato de texto (por ejemplo, "192.168.1.1").
getHostName()	Obtiene el nombre de host asociado a una dirección IP.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Ventajas y Desventajas

Ventajas de la Programación con Sockets en Java

- Las aplicaciones son más ordenadas y limpias en Java que en C o C++:En comparación con lenguajes como C o C++, las aplicaciones escritas en Java requieren menos líneas de código, lo que facilita su mantenimiento y comprensión.
- Cada línea de código puede explicarse fácilmente a programadores principiantes, ya que Java tiene una sintaxis más intuitiva y una mayor cantidad de abstracciones útiles. Esto reduce la complejidad, especialmente en programas relacionados con redes.

Desventajas de la Programación con Sockets en Java

- Java no expone toda la gama de posibilidades de los sockets al desarrollador:Aunque Java facilita enormemente el trabajo con sockets, esta facilidad viene con ciertas limitaciones.
  Los desarrolladores no tienen acceso directo a algunas funcionalidades avanzadas o específicas de los sockets, que podrían estar disponibles en otros lenguajes como C o C++.
  Por ejemplo, algunas configuraciones de bajo nivel, como el ajuste fino del comportamiento de los sockets (buffers personalizados, opciones avanzadas de control de tráfico, etc.), no están disponibles o son más difíciles de implementar en Java.
- Java abstrae la complejidad de la capa de transporte de sockets:Toda la complejidad relacionada con la capa de transporte de sockets (como establecer conexiones TCP o UDP) está oculta "bajo el capó".
  Esto significa que los desarrolladores no tienen que preocuparse por los detalles internos de cómo funciona la red o cómo se gestionan las capas de transporte.
  Los desarrolladores pueden centrarse únicamente en la lógica de su aplicación, sin necesidad de profundizar en la gestión manual de conexiones o en las peculiaridades del sistema operativo.
- Java no depende de código nativo:A diferencia de algunos lenguajes de programación que dependen de bibliotecas o funciones específicas de un sistema operativo, Java es completamente independiente de la plataforma.
  Esto permite que los programas desarrollados con sockets en Java puedan comunicarse a través de redes (incluido Internet) en diferentes plataformas sin problemas de compatibilidad.
  Gracias a la Máquina Virtual de Java (JVM), el mismo programa puede ejecutarse en Windows, Linux, macOS o cualquier otro sistema operativo compatible con Java.

La programación con sockets en Java es una solución poderosa y eficiente para desarrollar aplicaciones de red, gracias a su facilidad de uso, abstracción de complejidad y portabilidad. Sin embargo, puede no ser la opción ideal si se necesita un control profundo y detallado sobre el comportamiento de los sockets. Por esta razón, Java es más adecuado para aplicaciones estándar o

de alto nivel, mientras que lenguajes como C o C++ pueden ser preferidos para tareas más especializadas.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Sockets TCP

Los sockets permiten la comunicación entre procesos de diferentes equipos de una red. Un socket, es un punto de información por el cual un proceso puede recibir o enviar información.

Tal y como hemos visto anteriormente, existen dos tipos de protocolos de comunicación: TCP o UDP. En el protocolo TCP es un protocolo orientado a la conexión que permite que un flujo de bytes originado en una máquina se entregue sin errores en cualquier máquina destino Por otro lado, el protocolo UDP, no orientado a conexión, se utiliza para comunicaciones donde se prioriza la velocidad sobre la pérdida de paquetes.

A la hora de crear un socket hay que tener claro el tipo de socket que se quiere crear (TCP o UDP). En esta sección vamos a aprender a utilizar los sockets TCP y en el siguiente punto veremos los sockets UDP.

Al utilizar sockets TCP, el servidor utiliza un puerto por el que recibe las diferentes peticiones de los clientes. Normalmente, el puerto del servidor es un puerto bajo [1-1023].



Cuando el cliente realiza la conexión con el servidor, a partir de ese momento se crea un nuevo socket que será en el encargado de permitir el envío y recepción de datos entre el cliente/servidor. El puerto se crea de forma dinámica y se encuentra en el rango 49152 y 655335. De ésta forma, el puerto por donde se reciben las conexiones de los clientes queda libre y cada comunicación tiene su propio socket.



El paquete java.net de Java proporciona la clase Socket que permite la comunicación por red. De forma adicional, java.net incluye la clase ServerSocket, que permite a un servidor escuchar y recibir las peticiones de los clientes por la red. Y la clase Socket permite a un cliente conectarse a un servidor para enviar y recibir información.

A continuación se estudian las llamadas más importantes de Socket en java. Si deseas complementar dicha información, en el siguiente enlace puedes ver la documentación oficial de Sockets en java.



Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Programación de socket

Los sockets proporcionan el mecanismo de comunicación entre dos computadoras que usan TCP. Un programa cliente crea un socket en su extremo de la comunicación e intenta conectar ese socket a un servidor. El cliente debe conocer dos datos la dirección IP del servidor y el número de puerto.

Cuando se realiza la conexión, el servidor crea un objeto de socket en su extremo de la comunicación. El cliente y el servidor ahora pueden comunicarse escribiendo y leyendo desde el socket.

La clase java.net.Socket representa un socket y la clase java.net.ServerSocket proporciona un mecanismo para que el programa del servidor escuche a los clientes y establezca conexiones con ellos.

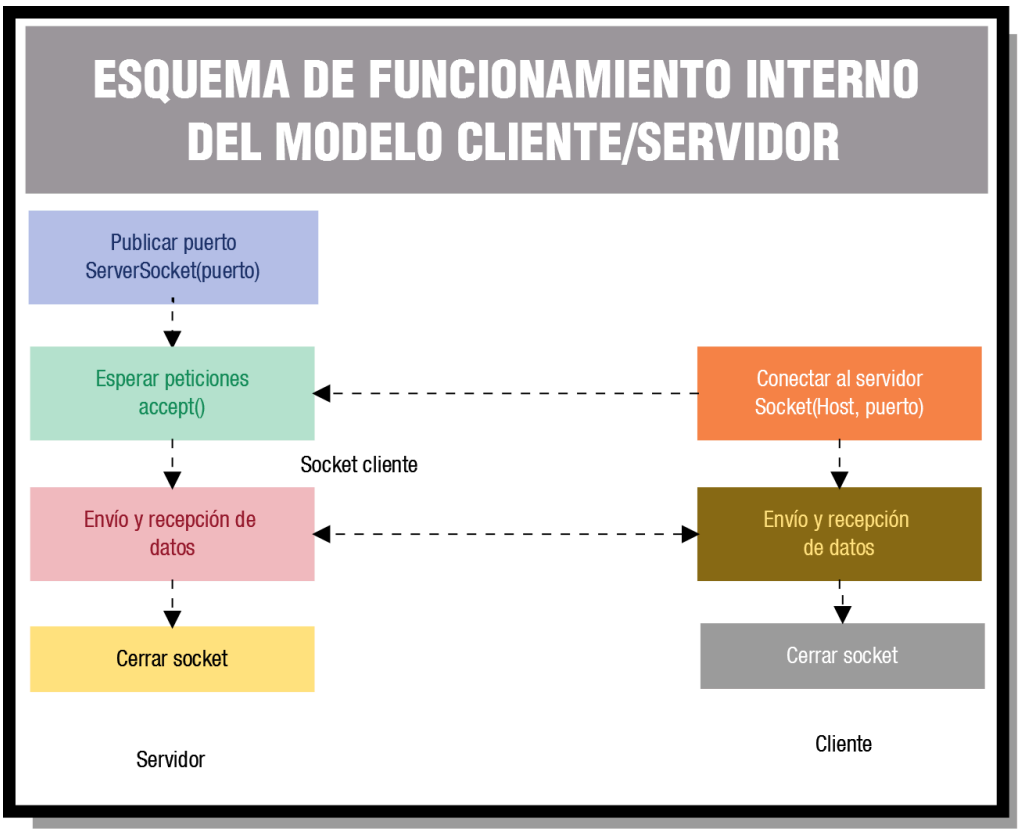
- Los siguientes pasos ocurren cuando se establece una conexión TCP entre dos computadoras usando sockets:El servidor crea una instancia de un objeto ServerSocket, que indica el número de puerto donde se va a producir la comunicación.
- El servidor invoca el método accept() de la clase ServerSocket. Este método espera hasta que un cliente se conecta al servidor en el puerto dado.
- Después de que el servidor está esperando, un cliente crea una instancia de un objeto Socket, especificando el nombre del servidor y el número de puerto para conectarse.
- El constructor de la clase Socket intenta conectar al cliente con el servidor especificado y el número de puerto. Si se establece la comunicación, el cliente ahora tiene un objeto Socket capaz de comunicarse con el servidor.
- En el lado del servidor, el método accept() devuelve una referencia a un nuevo socket en el servidor que está conectado al socket del cliente.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Servidor

Los pasos que realiza el servidor para realizar una comunicación son:

- Publicar puerto. Se utiliza el comando ServerSocket indicando el puerto por donde se van a recibir las conexiones.
- Esperar peticiones. En este momento el servidor queda a la espera a que se conecte un cliente. Una vez que se conecte un cliente se crea el socket del cliente por donde se envían y reciben los datos.
- Envío y recepción de datos. Para poder recibir/enviar datos es necesario crear un flujo (stream) de entrada y otro de salida. Cuando el servidor recibe una petición, éste la procesa y le envía el resultado al cliente.
- Una vez finalizada la comunicación se cierra el socket del cliente.



Para publicar el puerto del servidor se utiliza la función ServerSocket a la que hay que indicarle el puerto a utilizar. Su estructura es:

```
ServerSocket skServidor = new ServerSocket(Puerto);
```

Una vez publicado el puerto, el servidor utiliza la función accept() para esperar la conexión de un cliente. Una vez que el cliente se conecta, entonces se crea un nuevo socket por donde se van a realizar todas las comunicaciones con el cliente y servidor.

```
Socket sCliente = skServidor.accept();
```

Una vez recibida la petición del cliente el servidor se comunica con el cliente a través de streams de datos que veremos en el siguiente punto.

Finalmente, una vez terminada la comunicación se cierra el socket de la siguiente forma:

```
sCliente.close();
```

A continuación se muestra el código comentado de un servidor



Este ejemplo muestra un servidor básico en Java que escucha en un puerto específico y acepta una conexión de cliente. Una vez que el cliente se conecta, el servidor cierra la conexión.

```
import java.io.* ;
import java.net.* ;

class Servidor {
    static final int Puerto = 2000;

    public Servidor() {
        try {
            // Inicio la escucha del servidor en un determinado puerto
            ServerSocket skServidor = new ServerSocket(Puerto);
            System.out.println("Escucho el puerto " + Puerto);

            // Espero a que se conecte un cliente y creo un nuevo socket para
el cliente
            Socket sCliente = skServidor.accept();

            // ATENDER PETICIÓN DEL CLIENTE

            // Cierro el socket
            sCliente.close();
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }
    }

    public static void main(String[] arg) {
```

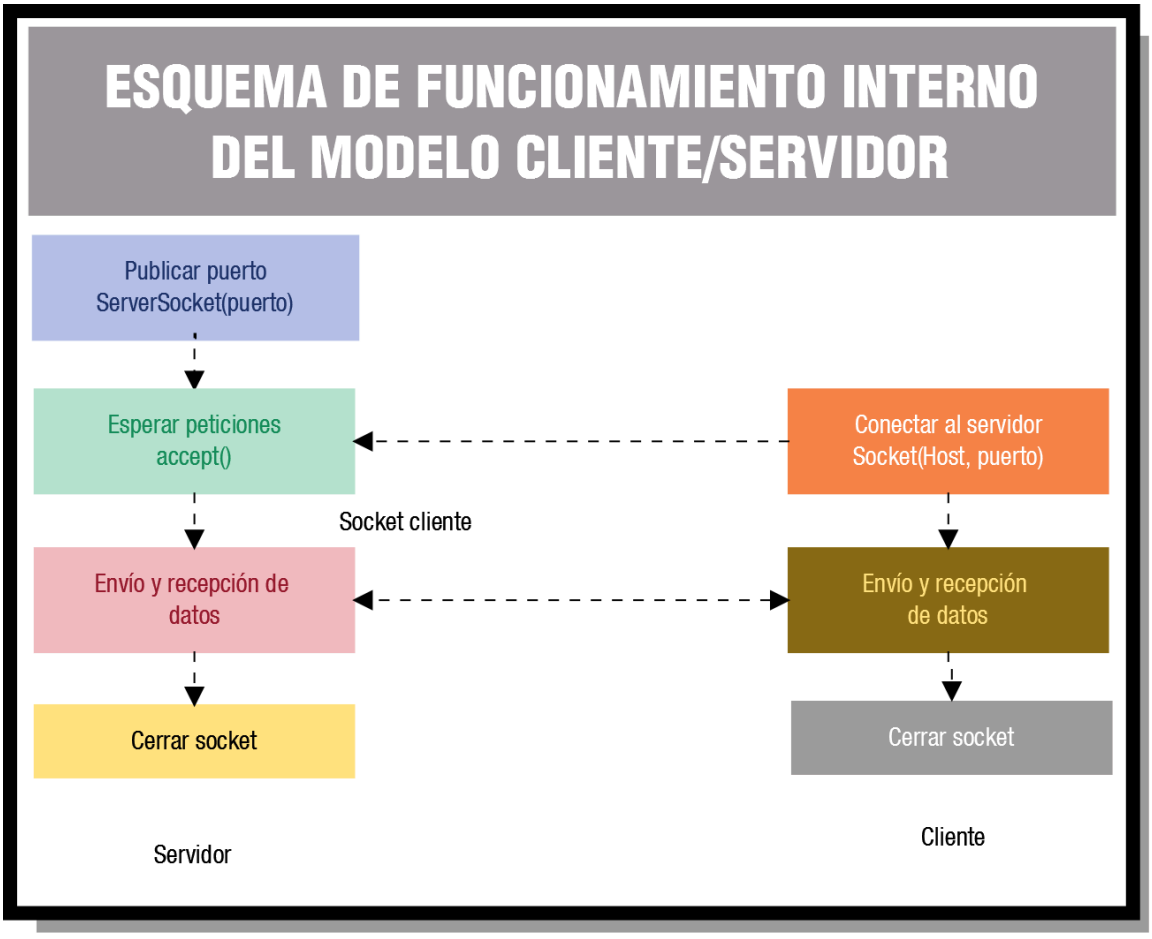
```
        new Servidor();
    }
}
```

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Cliente

Los pasos que realiza el cliente para realizar una comunicación son:

- Conectarse con el servidor. El cliente utiliza la función Socket para indicarse con un determinado servidor a un puerto específico. Una vez realizada la conexión se crea el socket por donde se realizará la comunicación.
- Envío y recepción de datos. Para poder recibir/enviar datos es necesario crear un flujo (stream) de entrada y otro de salida.
- Una vez finalizada la comunicación se cierra el socket.



Para conectarse a un servidor se utiliza la función Socket indicando el equipo y el puerto al que desea conectarse. Su sintaxis es:

```
Socket sCliente = new Socket( Host , Puerto );
```

donde Host es un string que guarda el nombre o dirección IP del servidor y Puerto es una variable del tipo int que guarda el puerto. "

Si lo prefiere también puede realizar la conexión directamente:

```
Socket sCliente = new Socket("192.168.1.200", 1500);
```

Una vez establecida la comunicación, se crean los streams de entrada y salida para realizar las diferentes comunicaciones entre el cliente y el servidor. En el siguiente apartado veremos la creación de streams.

Finalmente, una vez terminada la comunicación se cierra el socket de la siguiente forma:

```
sCliente.close();
```

A continuación se muestra el código comentado de un servidor.



Este ejemplo muestra un cliente básico en Java que se conecta a un servidor en un puerto específico y luego cierra la conexión.

```java
import java.io.* ;
import java.net.* ;

class Cliente {
    static final String Host = "localhost";
    static final int Puerto = 2000;

    public Cliente() {
        try {
            // Me conecto al servidor en un determinado puerto
            Socket sCliente = new Socket(Host, Puerto);

            // TAREAS QUE REALIZA EL CLIENTE

            // Cierro el socket
            sCliente.close();
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }
    }

    public static void main(String[] arg) {
        new Cliente();
    }
```

```
}
```

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Flujo de Entrada y de Salida

Una vez establecida la conexión entre el cliente y el servidor se inicializa la variable del tipo Socket que en el ejemplo se llama sCliente. Para poder enviar o recibir datos a través del socket es necesario establecer un stream (flujo) de entrada o de salida según corresponda.

Continuando con el ejemplo anterior, a continuación se va a establecer un stream de salida llamado flujo_salida.

```
OutputStream aux = sCliente.getOutputStream();

DataOutputStream flujo_salida= new DataOutputStream( aux );
```

o lo que es lo mismo,

```
DataOutputStream flujo_salida= new DataOutputStream(sCliente.getOutputStream());
```

A partir de éste momento puede enviar información de la siguiente forma:

```
flujo_salida.writeUTF( "Enviar datos");
```

De forma análoga, puede establecer el stream de entrada de la siguiente forma:

```
InputStream aux = sCliente.getInputStream();

DataInputStream flujo_entrada = new DataInputStream( aux );
```

o lo que es lo mismo,

```
DataInputStream flujo_entrada = new DataInputStream(sCliente.getInputStream());
```

A continuación se muestra una forma cómoda de recibir información:

```
String datos=new String();

Datos=flujo_entrada.readUTF();
```

Además de utilizar las funciones writeUTF y readUTF es posible recibir información de otras formas. Para más información consulte los siguientes enlaces.

Data Input Stream.

Data Output Stream.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Ejemplo

Para continuar con el ejemplo anterior y poder utilizar los sockets para enviar información vamos a realizar un ejemplo muy sencillo en el que el servidor va a aceptar tres clientes (de forma secuencial no concurrente) y le va a indicar el número de cliente que es.

Servidor.java



Este ejemplo muestra un servidor en Java que atiende a tres clientes de manera secuencial. A cada cliente le envía un mensaje de bienvenida y luego cierra la conexión.

```
import java.io.* ;
import java.net.* ;

class Servidor {
    static final int Puerto = 2000;

    public Servidor() {
        try {
            ServerSocket skServidor = new ServerSocket(Puerto);
            System.out.println("Escucho el puerto " + Puerto);

            for (int nCli = 0; nCli < 3; nCli++) {
                Socket sCliente = skServidor.accept();
                System.out.println("Sirvo al cliente " + nCli);

                OutputStream aux = sCliente.getOutputStream();
                DataOutputStream flujo_salida = new DataOutputStream(aux);
                flujo_salida.writeUTF("Hola cliente " + nCli);

                sCliente.close();
            }
            System.out.println("Ya se han atendido los 3 clientes");
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }
    }

    public static void main(String[] arg) {
```

```
        new Servidor();
    }
}
```

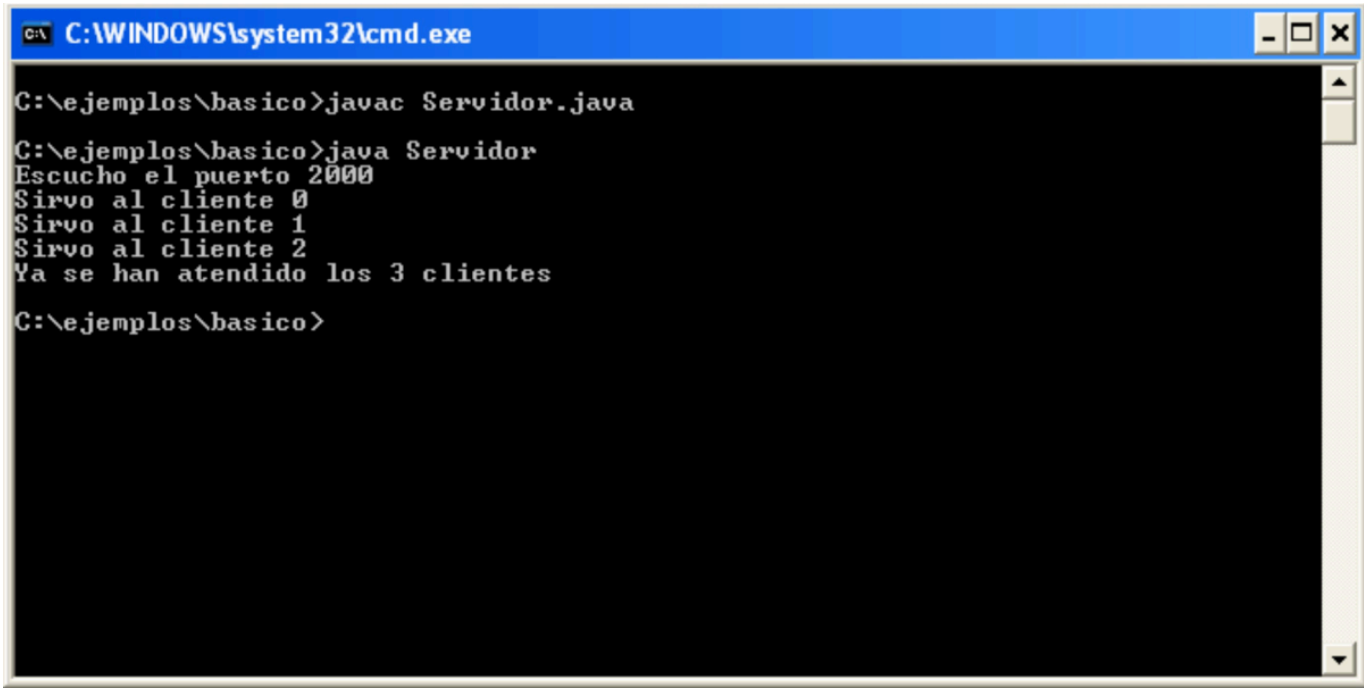
Para compilar el programa ejecuta:

javac Servidor.java

y para ejecutarlo ejecuta

java Servidor

En la siguiente figura se muestra al servidor como ha procesado las solicitudes de tres clientes.



De la misma forma, creamos el cliente con el siguiente código:

Cliente.java



Este ejemplo muestra un cliente en Java que se conecta a un servidor, recibe un mensaje y luego cierra la conexión.

```
import java.io.*;
import java.net.*;

class Cliente {
    static final String HOST = "localhost";
```

```
    static final int Puerto = 2000;

    public Cliente() {
        try {
            Socket sCliente = new Socket(HOST, Puerto);

            InputStream aux = sCliente.getInputStream();
            DataInputStream flujo_entrada = new DataInputStream(aux);

            System.out.println(flujo_entrada.readUTF());
            sCliente.close();
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }
    }

    public static void main(String[] arg) {
        new Cliente();
    }
}
```

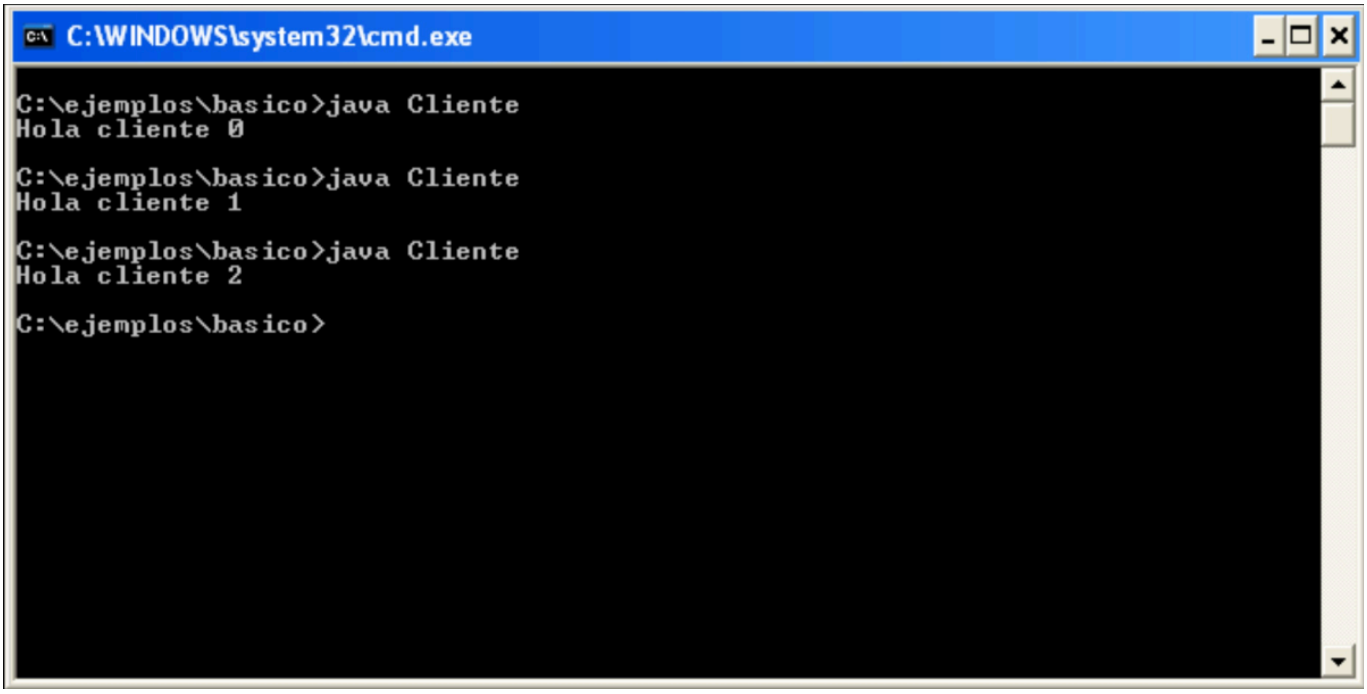
Para compilar el programa ejecuta:

javac Cliente.java

y para ejecutarlo ejecuta

java Cliente

Como puede ver en la siguiente figura, al ejecutar el cliente éste se conecta al servidor y muestra el mensaje que le envía el servidor.



Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Ejemplo

Debido a la naturaleza de bajo nivel de los sockets, se pueden conectar servidores y clientes escritos en distintos lenguajes, permitiendo una comunicación independiente de la plataforma y del lenguaje.

En el siguiente ejemplo el servidor está escrito en Java y el cliente en Python, intercamiando un byte representando un número entero.



Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Sockets UDP

En el caso de utilizar sockets UDP no se crea una conexión (como es el caso de socket TCP) y básicamente permite enviar y recibir mensajes a través de una dirección IP y un puerto. Estos mensajes se gestionan de forma individual y no se garantiza la recepción o envío del mensaje como si ocurre en TCP.

Para utilizar sockets UDP en java tenemos la clase DatagramSocket y para recibir o enviar los mensajes se utiliza clase DatagramPacket. Cuando se recibe o envía un paquete se hace con la siguiente información: mensaje, longitud del mensaje, equipo y puerto.

DatagramSocket.

DatagramPacket.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Receptor

En el caso de querer iniciar el socket en un determinado puerto se realiza de la siguiente forma:

```
DatagramSocket sSocket = new DatagramSocket(puerto);
```

Una vez iniciado el socket ya estamos en disposición de recibir mensajes utilizando la clase DatagramPacket. Cuando se recibe o envía un paquete se hace con la siguiente información: mensaje, longitud del mensaje, equipo y puerto.

A continuación se muestra un código de ejemplo para recibir un mensaje:

```
byte [] cadena = new byte[1000] ;
```

```
DatagramPacket mensaje = new DatagramPacket(cadena, cadena.length);

sSocket.receive(mensaje);
```

Una vez recibido el mensaje puede mostrar su contenido de la siguiente forma:

```
String datos=new String(mensaje.getData(),0,mensaje.getLength());

System.out.println("\tMensaje Recibido: " +datos);
```

Finalmente, una vez terminado el programa cerramos el socket:

```
sSocket.close();
```

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Emisor

Por otro lado, para realizar una aplicación emisora de mensajes UDP debe inicializar primero la estructura DatagramSocket.

```
DatagramSocket sSocket = new DatagramSocket();
```

Ahora debe crear el mensaje del tipo DatagramPacket al que debe indicar:

- Mensaje a enviar.
- Longitud del mensaje.
- Equipo al que se le envía el mensaje.
- Puerto destino.

A continuación se muestra un ejemplo para crear un mensaje:

```
DatagramPacket mensaje = new
DatagramPacket(mensaje,longitud_mensaje, Equipo, Puerto);
```

Para obtener la dirección del equipo al que se le envía el mensaje a través de su nombre se utiliza la función getByName de la clase InetAddress de la siguiente forma

```
InetAddress Equipo = InetAddress.getByName("localhost");
```

Una vez creado el mensaje lo enviamos con la función send():

```
sSocket.send(mensaje);
```

Finalmente, una vez terminado el programa cerramos el socket:

```
sSocket.close();
```

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Ejemplo

A continuación, para aprender a programar Sockets UDP se va a realizar un ejemplo sencillo donde intervienen dos procesos:

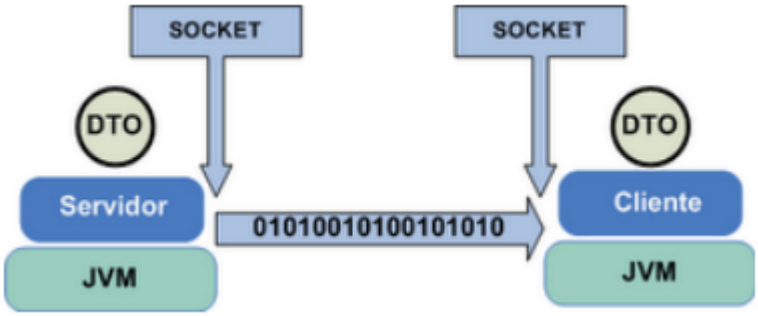
- ReceptorUDP. Inicia el puerto 1500 y muestra en pantalla todos los mensajes que llegan a él.
- EmisorUDP. Permite enviar por líneas de comandos mensajes al receptor por el puerto 1500.



Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Envío de objetos

Los stream que se usan para intercambiar datos entre programas cliente y servidor soportan la transferencia de objetos.



Las clases ObjectInputStream y ObjectOutputStream nos permiten enviar objetos a través de sockets TCP. Utilizaremos los métodos readObject() para leer el objeto del stream y writeObject() para escribir el objeto al stream. A partir de un socket,
Socket socket = new Socket(host,puerto);
Para preparar el flujo de salida para escribir objetos escribimos:
ObjectOutputStream outObjeto = new ObjectOutputStream(socket.getOutputStream());
Para preparar el flujo de entrada para leer objetos usaremos:
ObjectInputStream inObjeto = new ObjectInputStream(socket.getInputStream());

Código Java

```
import java.io.Serializable; public class Alumno implements Serializable {
private String nombre; private int edad; public Alumno(String n, int e) {
this.nombre = n; this.edad = e; } }
```

import java.io.Serializable; public class Alumno implements Serializable { private String nombre; private int edad; public Alumno(String n, int e) { this.nombre = n; this.edad = e; } }

Las clases a las que pertenecen estos objetos deben implementar la interfaz Serializable.

Enviar un objeto

Alumno alum=new Alumno(“Amadeo”,21);
outObjeto.writeObject(alum);
Leer un objeto
Alumno alum=(Alumno)inObjeto.readObject();

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Multiconexiones

Los sockets en Java, combinados con hilos, permiten crear aplicaciones de red que pueden manejar múltiples conexiones simultáneamente. Voy a explicarte los conceptos clave y mostrarte un ejemplo práctico.

Conceptos Básicos:

1. Socket de Servidor: Un socket que escucha conexiones entrantes
2. Socket de Cliente: Un socket que se conecta a un servidor
3. Hilos: Permiten manejar múltiples conexiones de forma concurrente

Ejemplo Práctico: Vamos a crear un ejemplo simple de un servidor de chat que usa sockets y hilos para manejar múltiples clientes.



Análisis de las Clases:

1. ChatServer:
    - Clase principal que inicia el servidor
    - Crea una instancia de ServerManager y lo arranca
2. ServerManager:
    1. Gestiona la creación del ServerSocket
    2. Acepta conexiones de clientes
    3. Usa un ExecutorService para manejar hilos de manera eficiente
    4. Tiene métodos para iniciar y apagar el servidor
3. ClientRegistry:
    1. Gestiona el registro de clientes conectados
    2. Proporciona métodos para añadir/eliminar clientes
    3. Implementa la difusión de mensajes entre clientes
    4. Usa CopyOnWriteArrayList para seguridad en hilos
4. ClientHandler:
    1. Maneja la comunicación con un cliente individual

2. Implementa Runnable para ejecutarse en un hilo
3. Gestiona el ciclo de vida de la conexión de un cliente
4. Métodos para enviar mensajes y desconectar

5. ChatClient (opcional):
    1. Cliente de ejemplo para probar el servidor
    2. Usa un hilo separado para recibir mensajes
    3. Permite al usuario enviar mensajes

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0

Ejercicios

EJERCICIO 1

Escribe un programa en java, “ClienteTCP1.java”, que sea capaz de aceptar dos parámetros de entrada en línea de órdenes para establecer una conexión: el nombre de un servidor y un número de puerto al que conectar en ese servidor. El programa debe visualizar un mensaje por pantalla indicando si la conexión se ha establecido o no. En caso de éxito debe mostrar también el número de puerto y el nombre del servidor con el que ha conectado, y en caso de fallo indicar el motivo: “Nombre de servidor desconocido” o “No es posible realizar conexión”, dependiendo el tipo de excepción ocurrida (ver notas en página siguiente).

EJERCICIO 2

Hacer un servidor Java que devuelva secuencialmente el siguiente insulto en su lista de insultos (InsultServer). El protocolo es:

```
MENSAJE: INSULTAME,

respuesta: INSULTOi.
```

El servidor debe soportar varios clientes (multi-hilo). Testearlo con algún cliente Java. Testearlo con el servidor de insultos de tu compañero. El puerto del servidor de insultos es el 1234.

EJERCICIO 3

Hacer un servidor multihilo en Java para la obtención y listado de nombres de ficheros en un directorio. El protocolo es:

```
MENSAJE: LISTALL.

RESPUESTA: Listado de todos los ficheros separados por espacios.

MENSAJE: EXIST.

RESPUESTA: YES|NO.
```

El puerto utilizado será el 1235.

EJERCICIO 4

Hacer un programa servidor (KeyServer) que devuelva la palabra clave de nuestro grupo. Protocolo: MENSAJE: CLAVE RESPUESTA: clave. Arrancarlo en un puerto entre el 1024 y el 65000.

EJERCICIO 5

Un cliente-servidor de chat que sea capaz de recibir conexiones de múltiples conexiones de clientes simultáneas, y que cada mensaje que mande un cliente lo envíe a todos los demás clientes.

EJERCICIO 6

Desarrollar una aplicación en Java que permita leer un fichero de texto ubicado en otro ordenador a través de sockets. Los pasos del proceso serán los siguientes:

- Programa cliente: solicita al usuario el nombre de un fichero incluyendo su ruta completa dentro del sistema de archivos del servidor.
- Programa cliente: envía el nombre y la ruta del fichero al servidor.
- Programa servidor: lee el contenido del fichero y se lo envía al cliente.
- Programa cliente: muestra el contenido por la pantalla.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0